OpenVAF — Verilog-A LRM Compliance

Clause-by-clause coverage of the Verilog-A language (VAMS LRM 2023, Annex C)

Ngspice + OpenVAF Enhancements project

August 2026

Contents

OpenVAF — Verilog-A LRM Compliance

How the openvaf-r compiler in this repository covers the Verilog-A language, clause by clause against the Verilog-AMS Language Reference Manual (the 2023 edition, `docs/VAMS-LRM-2023.pdf`; Verilog-A is its Annex C analog-only subset). Every claim in this document is backed by a committed, executable check: the code examples are drawn from the verify suites under `examples/`, which compile each construct with the committed compiler, simulate it with the committed ngspice, and compare the numbers against closed-form expectations on every regression run.

Status marks used throughout:

Mark	Meaning
✓	fully implemented and runtime-verified
⚠	implemented with LRM-sanctioned fallback semantics, or a documented deviation
×	out of scope: a Verilog-AMS (mixed-signal) feature outside Annex C

A one-page summary table closes the document. Companion references: the User Handbook (task-oriented), the openvaf change report (where each capability was implemented and why), and the per-enhancement write-ups in `enhancements_doc/`.

---

1. Compilation model and scope (LRM 1, Annex C)

OpenVAF compiles one Verilog-A source file (with its `` `include `` closure) into an OSDI shared object: analytic residuals and Jacobians are generated by automatic differentiation, so every construct below is "supported" only if it produces correct derivatives as well as correct values — a stricter bar than parsing. Hierarchy is resolved by a compile-time elaboration pass (§7), after which the pipeline sees a single flat module.

Annex C draws the language boundary this project targets: the analog subset. Digital and mixed-signal constructs (`reg`/`wire` logic, digital `always`/`initial`, connect modules, `` `default_discipline ``, discrete-domain disciplines) are × out of scope by definition, and the compiler rejects them with located diagnostics rather than accepting them silently — e.g. `domain discrete` on a discipline with natures is a proper two-label error citing LRM 3.6.2.2.

2. Lexical conventions (LRM 2) — ✓

Identifiers, including escaped identifiers (LRM 2.7.1):

```
electrical \node-1a ;          // any printable characters until whitespace
parameter real \my.gain = 2.0;
```

Numbers (LRM 2.5): decimal integers, reals with exponents, SI scale suffixes (`1k`, `2.5u`, `10n`), and based integer literals in all four bases with optional size and sign, underscores as separators:

```
integer a, b, c, d;
analog begin
    a = 8'hFF;          // sized hex          -> 255
    b = 'sb1010_1100;   // signed binary with separators
    c = 'o17;           // octal              -> 15
    d = 4'd12;          // sized decimal
end
```

The don't-care digits `x`/`z`/`?` are additionally accepted in the power-of-two bases when the literal is a `casex`/`casez` item (§5.4); anywhere else they are a located error rather than silent zeros.

The three-token forms of LRM 2.6.1 work as of E-515: the digits may be separated from the base by white space and each token may come from a macro — `5 'D 3` (the LRM's own Example 2), `12'b 0011_0101_0001`, `` `SZ'hFF ``, `'h 837FF`, `8'sh FF` — evaluated in the `lrmlex` suite next to the contiguous forms. A spaced literal whose digits are illegal for its base is a located error. Two illegal forms that used to compile in silence are errors now: `1.` (LRM 2.6.2 requires a digit on each side of the decimal point) and a string spanning a raw newline (LRM 2.7). A backslash immediately before the newline is a line continuation instead: the backslash–newline pair contributes nothing to the string (SystemVerilog 5.9 semantics and de-facto Verilog-A practice — BSIM4 splits its long `$strobe` messages exactly this way), so only an unescaped newline is refused. `assert`, `root` and `do` — none of them reserved by Annex B — are legal identifiers again (`do` contextually, so do-while still parses), and attribute instances parse in the expression positions of LRM 2.9, with duplicate attributes resolving to the last value as required.

Strings (LRM 2.6.3) with the full escape set — `\n`, `\t`, `\\`, `\"`, and octal `\ddd` — processed by a single-pass unescaper (a sequential-replacement implementation corrupted overlapping escapes and was rewritten). Comments both styles, including the edge case of a `//` comment at end-of-file without a trailing newline (formerly a lexer hang). Attributes `(* ... *)` parse everywhere the LRM allows and carry simulator-facing metadata (§9.6).

3. Data types (LRM 3)

3.1 Value types — ✓

`integer` (32-bit), `real` (IEEE double), and `string` variables, with genuine persistence across evaluations for both numeric types (a Verilog-A module variable keeps its value between solver calls, the foundation of event counters and iterative models):

```
integer count;              // persists across evaluations
analog begin
    @(cross(V(in) - 0.5, +1))
        count = count + 1; // genuinely accumulates
end
```

Uninitialized strings default to `""`; declaration initializers work on scalars and (multi-dimensional) arrays, evaluating once at setup when parameter-only:

```
real x = 2.0 * gain;
real taps[0:3] = '{0.1, 0.2, 0.3, 0.4};
```

A string literal assigned to an `integer` converts per LRM 3.3: the characters are treated as a packed byte vector, right-justified and truncated on the left / zero-filled to 32 bits (`"A"` is 65, `"AB"` is 16706 = 0x4142, `"ABCDE"` keeps its last four bytes). Only the *literal* converts — a string value still cannot be assigned to an integral type, exactly as the LRM words it.

3.2 Parameters (LRM 3.4) — ✓

Ranges and exclusions enforced on user-given values; `localparam` genuinely non-overridable; `aliasparam` including `$param_given` through the alias — with both of LRM 3.4.7's error rules enforced: naming the alias and its target on one `.model`/instance line is an error (in the ngspice netlist parser and in the OSDI `setup_model` path both), and referencing the alias anywhere in the module body is a targeted compile error; string parameters (usable in `case` dispatch); array parameters of any dimensionality, expanded to per-element OSDI parameters that are individually overridable from SPICE and overridable whole at instantiation (`leaf #(.w('{1,2,3})) l1(a,b);`, with the element count checked against the declaration):

```
parameter real r = 1k from (0:inf) exclude 1e6;
parameter integer sel = 0 from [0:15];
localparam real g = 1.0 / r;              // tracks r, not overridable
parameter real w[0:1][0:2] = '{'{1,2,3}, '{4,5,6}};
// SPICE: .model m dev(w[1][2]=9.5)
```

Two typing edges are deliberate deviations, documented in the handbook's limitations chapter: an untyped parameter's type is frozen at compile time from its default (`parameter untyped = 1;` is an integer parameter, and a real override is rounded — with an ngspice warning — where LRM 3.4.1 would re-type the parameter from the final overridden value; a compiled OSDI descriptor declares exactly one type per parameter). ⚠ And in the lenient direction, an untyped string or array parameter is accepted with its type inferred from the default, although LRM 3.4.1 makes the type mandatory for those two kinds.

One deliberate reading of the LRM, matching industry practice (the CMC convention): a parameter's default is exempt from its own range — compact models use out-of-range defaults as "feature disabled" markers — while every user-given value is fully validated. ⚠ documented in the handbook's limitations chapter.

Block-scoped parameters (LRM 6.3): a `parameter`/`localparam` declared inside a named `begin: label` block is a compile-time constant local to that block, read hierarchically and derivable from the module's parameters (E-87):

```
analog begin
   begin: s
      parameter real g2 = gain * gain;  // block param, from a module param
      real contrib = g2 + 1.0;
   end
   vout = s.contrib;                    // hierarchical read
end
```

Overriding a module parameter propagates into the block-scoped parameters that depend on it. The LRM's `#(.blk.p(4))` instance override of a block-scoped parameter is illegal (LRM 6.3.2) and rejected with a targeted diagnostic.

`paramset` blocks (a Verilog-AMS 6.4 feature adopted because real model libraries use them) work, including hidden-system-parameter assignments:

```
paramset fast_nmos nmos_va;
    .w = 2e-6; .l = 60e-9;
    .$mfactor = 4;
endparamset
```

3.3 Natures, disciplines, nets (LRM 3.5–3.7)

✓ Discipline/nature declarations with attribute access from expressions (`net.potential.abstol`); derived natures — `nature Xf : Flow` and deriving from a discipline's nature (`: electrical.flow`) — with full attribute inheritance; `ground` declarations in both orderings; vectored nets and ports with bit-selects; net initializers that become solver nodesets; `domain continuous`:

```
nature CurrentX : Current;   // derived nature, inherits abstol/units
    abstol = 1e-15;
endnature

electrical [3:0] bus;        // vectored net (range-then-name)
electrical data[0:7];        // vectored net (name-then-range, E-89)
electrical vout = 2.5;       // initializer -> OSDI nodeset (initial guess)
ground electrical gnd;
```

Both declaration orders are accepted for vectored nets and ports: the range-then-name form `electrical [3:0] bus;` / `input [3:0] bus;` and the name-then-range form `electrical bus[3:0];` / `input bus[3:0];` (E-89). A bit of a multi-bit input bus port reads its own terminal regardless of the port's position in the header — a bus port that is not the last port keeps its bits contiguous and in header order, so the simulator's positional terminal mapping stays correct (E-90). The name-then-range form also accepts a multi-name list with per-name widths (`input a[0:1], b[0:3], c;`, E-91). A net/port/array width may reference a parameter (`electrical [0:N-1] out;`, `real w[0:N-1];`, E-91): it is folded from the parameter's elaboration-time default and fixed at that value — a structural parameter, since the OSDI descriptor has one node count per module. Such a parameter is frozen to a **`localparam`** (E-92) so a netlist override cannot resize the compiled structure (or desync it from behavioural code); to change the width, edit the default and recompile. Any `localparam` (frozen width parameter or hand-written) is flagged non-settable in the OSDI descriptor, so ngspice warns rather than silently ignoring a netlist attempt to set it (E-93). A multi-dimensional vectored port (`in[0:2][0:1]`) is not supported in either declaration order.

Signal-flow disciplines (potential-only or flow-only) are fully usable, including probe-only branches — probing a branch that is never contributed to reads its true flow (an ideal ammeter) instead of the 0-and-open-circuit a naive topology gives.

Discipline compatibility implements all of LRM 3.11.1 (natures audit): a branch — declared or unnamed — between a conservative net and a signal-flow net of the same potential nature is legal (the LRM's own worked example declares `electrical` and `sig_flow_v` compatible: ”the nature for flow does not exist in sig_flow_v”), a natureless discipline is compatible with its whole domain, and a domainless one with everything. The previous rule demanded every nature binding both-present-or-both-absent, rejecting exactly those pairings — and the diagnostic's help text stated a units-only rule that was never the LRM's. A mixed branch takes the discipline that actually has the natures, so `I(br)` across `electrical`/`voltage` works. Genuinely incompatible pairings (`electrical` vs `rotational`) are still rejected.

Nature attribute forms are validated again (the checks existed but iterated the wrong accessor and never ran): `access = "SA"` or `access = 3.0` (LRM 3.6.1.2 requires a raw identifier) and a `ddt_nature`/`idt_nature` value that is not a nature name are located errors at the declaration, instead of being silently dropped — which used to leave the nature with no access function and only a baffling ”not found in the current scope” error at the first use. Two more 3.6.1.2 rules are warnings: a *derived* nature declaring `units` (the declared value is ignored — the parent's units always win — so saying nothing hid a modelling error), and an `idt_nature`/`ddt_nature` override unrelated (no shared base nature) to the link the parent uses.

Two deliberate deviations stand, now recorded here: ⚠ a base nature may omit the `abstol`/`access`/`units` attributes 3.6.1.2 requires (E-422's pinned decision — omission produces no wrong answer), and ⚠ a derived nature may declare its own access function (E-39 supports it on purpose: `nature Current2 : Current; access = J;` gives a working `J()` — the LRM calls this illegal). Two constructs are not supported and rejected with located diagnostics: vector branches (LRM 3.12's `branch (a,b) br1;` over whole vector nets — use per-bit branches `branch (a[i],b[i])`), and out-of-module discipline declarations (LRM 3.10's `electrical top.other.net;` — meaningless in a single-module OSDI compile).

The shipped **`constants.vams`** is the VAMS-2023 Annex D.2 file: defining `` `PHYSICAL_CONSTANTS_NIST2018 `` before the include selects the exact 2019-SI values (`P_Q` = 1.602176634e-19, `P_K` = 1.380649e-23, `P_H`, `P_EPS0`, and the measured `P_U0` = 1.25663706212e-6); the default without the define stays NIST1998 for backward compatibility, exactly as the 2023 LRM specifies. (The 2.4.0 file shipped before had no NIST2018 set, so opting in silently produced NIST1998 values.) The OSDI 0.4 nature descriptors are also exact now: every nature's `num_attr` claimed one attribute more than it owned (a consumer walking the range read the next nature's first attribute), verified fixed by a dlopen dump of the emitted `OSDI_NATURES` table.

Named port branches (LRM 3.7.2) work as of E-84 and carry the same defining equation as a direct port-

flow probe; contributing to one is a proper diagnostic (port branches are probe-only per the LRM):

```
branch (<p>) probe_p;          // named branch over port p
iprobe = I(probe_p);           // same value as I(<p>)
```

Hierarchical branch probes (E-86) read another instance's branches from outside it — named, unnamed, and port-branch forms, the last via a 0V ammeter synthesized at flattening so it reads that instance's own current rather than the node total:

```
V(top.a1.b)              // named branch of instance a1
V(top.d1.branch(va, vb)) // unnamed branch of instance d1
I(top.d1.branch(<p>))    // d1's own current into its port p
```

4. Expressions (LRM 4)

4.1 Operators and precedence (LRM 4.1, Table 4-2) — ✓

The full operator surface — arithmetic (incl. `**`), comparison, logical, bitwise, shifts, ternary (including over strings), concatenation `{a,b}` and replication `{n{x}}` — was audited operator-by-operator and level-by-level against Table 4-2 with self-checking bitmask modules whose failure modes are numerically visible. Two real defects found and fixed in the audits: unary `~` emitted arithmetic negate, and the constant folder disagreed with the runtime on `>>`; one precedence defect: `%` bound tighter than `*`/`/` (so `6*7%4` evaluated to 18 instead of 2). Associativity verified including `2**3**2 = 64` (left-associative per Verilog) and unary binding above `**` (`-2**2 = 4`). String comparison covers both equality (`==`/`!=`) and the relational operators (`<`/`<=`/`>`/`>=`), the latter a lexicographic (`strcmp`) comparison (E-106).

The arithmetic shifts **`<<<`/`>>>`** are a flagged extension: LRM 4.2.11 bars them from analog blocks, so every use draws a warning naming the rule — but they keep working, because `>>>` is Verilog's only spelling of a sign-extending right shift (`<<<` is identical to `<<`). ⚠ The case (in)equality operators **`===`/`!==`** (LRM 4.2.6) lex and evaluate with `==`/`!=` semantics — exact in a 2-state analog world, which has no x/z bits for them to distinguish (they previously died with a generic parse error that never named the operator). A compile-time shift distance outside 0..=31 is a warning plus the LRM-defined value — the distance is unsigned (LRM 4.2.11), so `1<<32` is 0 and `-8>>>34` is the sign fill, exactly what a run-time distance computes; it used to be a hard error, internally inconsistent with that run-time path. And integer `/` and `%` are now fully defined at the code-generation level: divide-by-zero yields 0 and `INT_MIN/-1` wraps to `INT_MIN`, where LLVM's raw `sdiv`/`srem` were undefined behaviour that could SIGFPE an x86 host the moment a runtime divisor crossed zero.

Same-node branch access is an error (LRM 4.4, Table 4-16): `V(a,a)` and `I(a,a)` no longer compile — "the operands of an expression shall be unique to define a valid branch", and the table marks both forms Error. `V(a,a)` used to compile with no diagnostic at all and silently read 0. A *named* degenerate branch declaration (`branch (a,a) b;`) keeps its E-414 warning: declaring one is survivable, accessing it is not.

`%` by a deck-supplied zero is a run-time error (LRM 4.2.4: "It shall be an error to pass zero (0) as the second argument to the modulus operator"): a literal zero divisor was already a compile error; a zero arriving through a model card now aborts the analysis with `OSDI(fatal) … %: the second operand (the modulus divisor) is zero` instead of a silent NaN and a generic convergence failure — the same three-route policy as the math builtins (E-509). A genuinely runtime divisor is deliberately left unguarded and evaluates to the defined 0.

4.2 Built-in and environment functions (LRM 4.3, 9.7) — ✓

`ln`/`log`/`exp`/`sqrt`/`pow`/`min`/`max`/`abs`/`floor`/`ceil`, trigonometric and hyperbolic families (integer `min`/`max`/`abs` correctly integer-typed; integer division truncates toward zero and real→integer conversion rounds to nearest, ties away from zero, per the LRM — the two are distinct rules; `ceil` of a runtime argument fixed, E-103), `$clog2` returning `ceil(log2 n)` (E-101), the conversion functions

`$rtoi` (real→integer, truncating toward zero — as distinct from the rounding implicit cast) and `$itor` (integer→real, E-104), and the environment: `$temperature` (kelvin — verified as the exact kT/q law across a −50…150 °C sweep), `$vt` and `$vt(T)` — computed from the 2019 exact SI k and q since the kernel audit, so `$vt(T)` equals `` `P_K*T/`P_Q `` exactly under `` `define PHYSICAL_CONSTANTS_NIST2018 `` (the shipped `constants.vams` still *defaults* to the NIST1998 set for LRM backward compatibility, so the two spellings differ by ~1 ppm under the default set — pick one constants set per model) — `$abstime`, `$realtime` (identical in the continuous-time analog context), `$simparam`, and `$simparam$str` serving `analysis_name`, `analysis_type`, `cwd` and `simulator` (Table 9-28's `module/instance/path` stay unserved with the honest warn-then-fatal: the channel carries no instance identity), plus `$param_given`, `$port_connected`, `$mfactor` and the position hidden parameters. `$realtobits`/`$bitstoreal` are a documented subset boundary: their 64-bit-vector operand type does not exist in analog Verilog-A, and the call is a clean resolution error.

`ln1p` and `expm1` (LRM Table 4-14 and Annex A.8.2) are present in both the bare and `$`-prefixed spellings, and lower to libm's `log1p`/`expm1` rather than to `ln(1+x)`/`exp(x)−1` — the precision near zero is the entire reason the standard lists them separately, and `ln1p(1e−18)` returns `1e−18` where the naive identity returns 0. They were added by E-458 and made *usable* only by E-510: the code generator asked for those two libm routines by name and the intrinsic registry declared neither, so every call whose argument was not constant-folded away crashed the compiler. The gap survived because the suite that introduced them tests `$ln1p(0.5)` — a literal, which folds before code generation, so the intrinsic is never emitted. `lrmintrin` now exercises both spellings with a parameter and with a probe.

Domain errors are refused, by whichever route the value arrives. An out-of-domain *constant* — `sqrt(−4)`, `ln(0)`, `asin(2)`, `acosh(0.5)`, `atanh(1)`, `pow` with a negative base and a fractional exponent — is a compile-time error naming the builtin, the value and the domain (E-455/479/489/508). The identical value supplied from a model card is an overridable `parameter`, which the compiler deliberately does not fold, and it used to reach the maths library untouched and return `nan`/`inf` silently; E-509 refuses it at run time with the same information. A genuine *run-time* argument is deliberately left alone: `sqrt(V(p,n))` legitimately sees a negative argument during Newton iteration, and refusing that would break working models. ⚠ That last route is a documented deviation from LRM 4.3.2's unqualified ”input values outside of the valid range for the operator shall report an error”. One scoping note on the claim above: `0**0` and `pow(0,0)` evaluate to 1.0 — the near-universal convention, and IEEE 1364's integer-power table says the same — although Table 4-14's `pow` domain (`x = 0` requires `y > 0`) reads the point out; the guards flag `0**negative` and a negative base with a fractional exponent, not `0**0`.

4.3 Analog operators (LRM 4.5) — ✓ (one documented deviation)

Every analog operator produces exact Jacobian contributions via autodiff. The full argument surface (trailing tolerances, optional arguments) was audited at runtime — compiling proves nothing about whether a trailing argument is honored.

```
// time derivative & integrals
I(p,n) <+ ddt(c * V(p,n));
phi = idtmod(K * V(vco), 0.0, 1.0);        // phase wrapping (VCO idiom)
x   = idt(V(in), 0.5, rst);                // reset form holds at ic

// delays and filters
V(out) <+ absdelay(V(in), 1n);
V(out) <+ transition(sel ? 1.0 : 0.0, 0, 10n, 5n);
V(out) <+ slew(V(in), 1e6, -1e6);          // LRM negative max_neg_rate
V(out) <+ laplace_np(V(in), '{1.0}, '{-6.283e3, 0.0});  // complex pairs
V(out) <+ zi_nd(V(in), '{0.5, 0.5}, '{1.0}, 1u);

// small-signal & events
```

```
gm = ddx(Id, V(g,s));                       // symbolic derivative
tc = last_crossing(V(in) - 0.5, +1);
V(out) <+ ac_stim("ac", 1.0, `M_PI/2);      // AC stimulus; phase in RADIANS (LRM
 ↪  4.6.3)
```

- `ddt`, `idt` (with initial conditions that survive into transient, and the assert/reset forms stable enough for relaxation oscillators), `idtmod` (integrates unbounded internally, wraps the returned value; a no-ic **`idtmod`** defaults its initial condition to 0 and pins the DC solution per 4.5.5's normative sentence — it used to demand external feedback like a no-ic `idt` and went *singular* without it; Table 4-19's "determined by the simulator" row reads the other way, and the paragraph's "shall" wins), `ddx` with respect to potentials *and flows* — including the unnamed-branch flow **`ddx(f, I(a,b))`** (Table 4-16 makes `I(n1,n2)` a branch flow, so it is inside 4.5.6's surface; verified `2·I` exact, the reversed orientation `I(b,a)` negated, and 0 when the flow is not a system unknown): ✓
- `absdelay` (+`maxdelay`), `transition` (3/4/5-argument, `` `default_transition `` defaults — including for an explicit zero rise/fall time, the LRM's own wording "specified or are equal to zero", which used to switch instantaneously; and with *no* directive, the 1- and 2-argument forms now take the LRM's "negligible, but non-zero" transition instead of an exact identity — DC well-posed), `slew` (honoring the LRM's *negative* `max_neg_slew_rate` — the sign defect that made it ignore its input was found in the audit): ✓
- `laplace_nd/np/zd/zp` via exact state-space realization and `zi_nd/np/zd/zp` via bilinear transform, both with complex pole/zero pairs per the LRM's (re, im) vector convention and parameter-dependent coefficients: ✓

  Three approximations in this group are deliberate and now stated plainly. ⚠ `transition`/`slew` are one rate-limited tracking loop at the fixed rate `1/trise`: exact for the comparator-style unit-amplitude input they are written for, while an amplitude-A step completes in `A·trise` (the LRM's per-transition scheduling, where every transition takes exactly `trise`, is not modeled) — and the small-signal transfer is a first-order lowpass with corner `1000/trise` rad/s rather than the LRM's approximate unity at all frequencies. ⚠ `zi_*` transient behavior is the continuous bilinear image of `H(z)`, not discrete-time sample-and-hold: a unity z-filter is a wire, no staircase exists, and the `tau`/`t0` arguments of the 6-argument form are accepted but ignored (true sampling needs simulator-side breakpoints). ⚠ A solution-dependent **`laplace_*`** coefficient TRACKS — LRM 4.5.14 classes the vectors as constant expressions whose dynamic value freezes at analysis start — and now draws a compile-time warning saying so (a dynamic `zi_*` constant slot stays refused outright, the over-strict-but-safe reading).

  Verified against closed-form `H(s)`: all four `laplace_*` forms agree with the analytic one-pole response to 4.7e-16 over four decades, a complex pair (`wn = 1/tau`, `Q = 1`) written as `zp`, `np` and `nd` coefficients agrees to 4.6e-15, and all four `zi_*` forms agree with the bilinear $H(z^{-1})$ to 1e-15 out to `wT = 1`.

  A complex root must be written together with its conjugate — the model's duty, not the compiler's. LRM 4.5.11.1 states it directly:

  > If a root is real, the imaginary part shall be specified as zero (0). If a root is complex, its conjugate shall also be present.

  The roots are expanded as $\prod(1 - s/r)$ exactly as the clause's formula shows (including its exception that a root of zero contributes `s` rather than `1 - s/r`), and the coefficients' imaginary parts cancel *because* the conjugate is there. Writing one root of a pair is therefore a model error, and it does not fail loudly: the imaginary parts no longer cancel, the real part is taken, and a different, physically real filter is built. A `Q = 1` pole pair written as the single root `'{-0.5/tau, 0.866/tau}` becomes one real pole at `2/tau` — a 20 kHz first-order rolloff in place of a 10 kHz resonant pair, with no diagnostic.

⚠ Open: that is a detectable mistake for constant roots, which is how they are almost always written, and nothing currently checks it. Recorded here rather than fixed.

- `last_crossing` with simulator-side waveform history: ✓ — and, per LRM 4.5.10, negative until the expression has actually crossed (E-514); it returned `0.0`, which a model testing `last_crossing(...) < 0` for *not yet* read as a crossing at t = 0. For an expression already above zero the zero-seeded crossing history also faked a crossing at the first accepted point, so both the sentinel and the history are now seeded from the operating point.

Analog-operator state at the start of a transient (E-514)

LRM 4.5.7 defines `absdelay` as `Output(t) = Input(max(t − td, 0))`, so before one delay has elapsed the output is `Input(0)` — the operating point, not zero. LRM 4.5.9 says `slew` *"returns the value of expr"* whenever the input is not moving faster than the rate limit, which a constant input never is. Both were violated the same way: the transient's state arrays were seeded with zero, so `slew`/`transition` ramped up from 0 at exactly the rate limit and `absdelay` read 0 for its whole first delay before stepping to the bias. Both now seed from the converged operating point. LRM 4.5.15's *"no state history prior to time t == 0"* is what `max(·, 0)` accommodates; it does not make the value zero.

The frozen-**`td`** rule is implemented now (analog-operators audit closing E-514's deferred item, by exactly the mechanism its analysis prescribed): with no `maxdelay`, *"the value of td when the absdelay() is first evaluated shall be used and any future changes to td shall be ignored"* — a per-slot flag in `OsdiAbsDelayInfo` marks the no-`maxdelay` form, and ngspice latches `td` at `MODEINITTRAN`, where the operating point has converged (the model-visible `IsInitialStep` flag still sees the zero initial guess, which is why compiled code cannot latch it). Verified on a ramp with `td = 1ms + 1ms·V(ctl)` stepping mid-run: the no-`maxdelay` form answers with the frozen 1 ms delay (4.99 V, previously 3.99), while the `maxdelay` form tracks as the LRM asks.

Two smaller honesty items from the same audit: ⚠ the `idt` assert/reset is a stiff first-order decay toward **`ic`** ($\tau = 10\ \mu s$; ~90 % of the deviation remains 1 µs into a reset, gone by ~5 τ) rather than the LRM's instantaneous return — E-52's deliberate choice, because the algebraic pin made the transient integrator see an impulse and self-resetting integrators ring; and ⚠ the **`abstol`**/nature tolerance arguments of `ddt`/`idt`/`idtmod` are parsed and validated (positive, a real nature) but do not influence simulator convergence tolerances — the OSDI ABI has no per-equation tolerance channel, so ngspice applies its global `abstol`/`reltol` to the implicit-equation unknowns.

- noise sources (LRM 4.6): `white_noise`, `flicker_noise`, `noise_table`, `noise_table_log` (inline or file data); operating-point-dependent factors and `ddt()`-shaped noise are exact with no extra matrix unknowns. Correlation follows the CALL (4.6.4.6, analysis-noise audit): one call's output routed into several contributions sums coherently as amplitudes — anti-phase uses cancel exactly — while two separate calls stay uncorrelated *even when they share a name*; the name is the 4.6.4.1 reporting label, and ngspice's contribution summary combines a label's power onto one vector. (E-42 first keyed correlation on the name string, which made same-labelled separate calls cancel; the audit moved the key to the call site, exported through the OSDI source name.) ✓. ⚠ `noise_table`'s array-parameter vector and string-parameter filename forms (4.6.4.3) are refused with located errors: the table is baked at compile time, and a deck-overridable parameter cannot feed it by construction — use a literal array/filename. `noise_table` interpolates the power piecewise-linearly over frequency (LRM 4.6.4.3) and `noise_table_log` interpolates log-log (`P = 10^(lerp of log10 p over log10 f)`, LRM 4.6.4.4) — both taking Hz input and clamping to the endpoint powers outside the tabulated range (Enhancement-109 corrected these; they had been a lin-log hybrid and a mis-keyed log-frequency input respectively): ✓

  ```
  I(a,b) <+ white_noise(4.0 * `P_K * $temperature / r, "thermal");
  I(d,s) <+ gm * white_noise(gamma_4kT, "induced");   // op-dependent factor
  ```

- `analysis("ac","tran",…)` variadic, with the AC/noise operating point counting as its parent

analysis per LRM 4.6.1 — and, since the analysis-noise audit, the full Table 4-22 static-phase detail: `analysis("ic")`/`analysis("static")` are 1 at the operating point that precedes a transient (they used to ride the first accepted timestep instead, so the 4.6.1 `if (analysis("ic"))` initial-condition idiom fired mid-transient), and `analysis("nodeset")` is 1 during the iterations in which the deck's `.nodeset` values are enforced (the flag existed and was never set). `ac_stim` activates against the RUNNING small-signal analysis: an `"ac"` stimulus no longer injects into a `.noise` gain solve (it poisoned the input-referred noise by exactly the injected stimulus), and `ac_stim("noise")` participates there, as 4.6.3's name matching requires: ✓

- `$limit` (built-in `"pnjlim"` and user functions — genuinely engages SPICE-style iteration limiting; an unknown function name falls back to no limiting with a warning per 9.17.3), `$bound_step` — with 9.17.2's smallest-active-bound rule enforced since the kernel audit: several calls in one evaluation used to leave the LAST one as the cap — and one ⚠ deliberate floor: a bound demanding more than 1e6 steps of the analysis window is overridden after a named warning (E-504; a device cannot demand an unbounded step count) — `$discontinuity(n)` (clamps the next step *and* makes the integrator bisect onto the event): ✓

- `$table_model` (LRM 9.21; an earlier revision of this document cited it as 9.19): 1-D piecewise-linear, N-dimensional multilinear, natural cubic-spline, and — since the kernel audit — closest-point lookup (`D`, with 9.21.4's farther-from-zero tie rule), all lowered to differentiable MIR so *all* partial derivatives feed the Jacobian (a table-based MOSFET gets exact gm and gds):

  ```
  I(d, s) <+ $table_model(V(g, s), V(d, s), "mos_iv.tbl", "1L");
  ```

  The kernel audit brought the whole 9.21 surface to the clause: default extrapolation is linear on both ends (Tables 9-31/9-32 — it used to clamp unless an `L` appeared); the control string's comma-separated sub-strings apply per dimension, outermost first (any code used to apply to every axis); up to two extrapolation characters set each end separately (`"1CL"`); **`E`** errors on extrapolation at run time; the **`;N`** dependent-column selector is honoured; data files may be the LRM 9.21.1 normative N+M-column isoline format — ragged isolines included (the LRM's own sample file interpolates correctly), with the project's self-describing N-D grid format still accepted as an extension — and 1-D data may be a pair of arrays (`'{xs}, '{ys}`) besides the interleaved layout. ⚠ Still refused with located errors and recorded here: quadratic splines (`2`), the ignore-column code (`I`), and the 9.21.5 whole-array N-dimensional data form (use the isoline file format); the runtime array-variable form supports `1`/`3` with same-both-ends `C`/`L` only.

- ⚠ **`limexp`** is implemented stateless (exact `exp` below the overflow threshold, tangent-continued above): a stateful previous-iterate limiting version was built and *reverted* because correct converged values under limiting require the simulator's limiting-RHS correction, which applies to circuit unknowns, not to `limexp`'s derived argument. `$limit` provides genuine iteration limiting. Documented decision with the failing evidence preserved.

- **`transition`** and **`slew`** reach their final value at `delay + trise`, at every speed — fixed in E-512, and worth recording here because this section previously carried it as a deviation.

  Both are one rate-limited tracking loop, `dy/dt = clamp(K·(x − y), −1/tfall, +1/trise)`. While the clamp is saturated that is an exact linear ramp at the LRM's rate; it releases once the remaining gap falls below `rate/K`, leaving a first-order tail with $\tau = 1/K$. `K` was a fixed 1e9 s^{-1}, so the released gap was `1/(K·trise)` and the linear fraction depended on how fast the edge was: at `trise = 3 ns` only 66.7% of the swing was a ramp and the output reached 0.8774 instead of 1.0, while at 3 µs it was correct. Refining the timestep 100× converged to 0.8776 — to the *wrong* value.

  `K` is proportional to the transition rate now, so the released gap is a fixed fraction at every speed, the endpoint error is 2e-5…4e-4 across five decades of rise time, and the settled value is exactly 1.0. `transedge` spans those five decades and checks the quarter points of the ramp; `defaulttransition` pinned only the 1 µs case, which is why the deviation went unseen for so long.

In DC the filter remains the LRM's static identity `y = x`, selected through the integration-enable parameter — that is what makes the operating point well-posed, and it is independent of the transient shape (E-47).

Out-of-domain operator arguments from the deck (E-504/E-506). The same literal-versus-deck split described for the built-in functions in §4.2 applies to the operators' own numeric arguments, and each case is resolved by what the argument *means* rather than by a blanket rule:

- a `zi_*` sampling period ≤ 0 inverts the bilinear map and reflects every pole across the imaginary axis; there is no honest value to project it onto, so the run time says what the compiler says and aborts, naming the value;
- a `laplace_*` leading denominator coefficient of zero is a division by zero and aborts the same way;
- an `idtmod` modulus ≤ 0 falls back to the unwrapped integral, because that is exactly what `idtmod` means with no modulus supplied — the model keeps running and the value stays finite;
- the `$rdist_*` family clamps onto the nearest distribution it can actually produce (a negative standard deviation behaves as 0), rather than returning the exact negation of the valid deviate as it once did.

Restrictions enforced as the LRM demands (4.5.1): analog operators inside loops or solution-dependent conditionals are rejected with diagnostics that name the actual construct and cite the clause. The conditional rule is applied as the LRM states it — only a condition that depends on an analog signal is refused, so the common idiom of gating an operator on a `parameter` (`if (SELFHEATING) ... ddt(...)`) compiles.

5. Analog behavior (LRM 5)

5.1 Analog blocks (LRM 5.2, 6.2) — ✓

Multiple `analog` and `analog initial` blocks per module execute as-if-concatenated in source order — verified including across instance flattening and with declarations interleaved between blocks.

5.2 Contribution statements (LRM 5.6) — ✓

Direct contributions with accumulation semantics, switch branches (the two kinds on mutually exclusive conditional paths — contributing both with no condition between them keeps only the last, which E-400 reports), indirect branch assignment (the LRM's ideal-op-amp construct), and port-flow probes:

```
V(out): V(inp, inn) == 0;      // indirect: drive out so V(inp,inn)=0
I(p,n) <+ V(p,n)/r;            // accumulates with other <+ statements
iport = I(<p>);                // total flow through port p (ideal probe)
branch (<p>) pb;               // named port branch: I(pb) == I(<p>)  (E-84)
```

`$port_connected` works on connected *and* unconnected ports of flattened instances (resolved per instance at elaboration time), and instantiating an undefined module is a hard error rather than a silent drop (both E-84). Bus actuals may be part-selects — positional, named, or width-1 onto a scalar port (E-85):

```
adc2 hi (out[3:2], in);        // positional slice
adc2 lo (.out(out[1:0]), .in(r)); // named slice
```

Indirect-assignment placement rules hold (behavior audit): LRM 5.6.7 forbids an indirect branch assignment in conditional or looping statements unless the controlling expression is constant, and 5.6.5 forbids contributions inside event controls — a guarded-off indirect assignment leaves its constraint as `0 = 0`, a singular matrix, so all three placements are compile errors (the constant-condition carve-out passes untouched). LRM 5.6.7.2's incompatibility rule is enforced in the same pass: a direct `<+` onto a branch that is also the target of an indirect assignment is an error naming both statements — it used

to compile and be silently absorbed by the implicit unknown. ⚠ The equality's left side accepts any real expression (`V(out): 2*V(a,b) + 1 == 1;`), a deliberate generalized-implicit- equation extension beyond Syntax 5-6's access-function-only LHS (the LRM's tolerance-selection intent is honored only for the access-function forms).

Hierarchical contributions create their own branch (LRM 5.6.8.1 with 5.5.4): `V(p, c1.mid) <+ 0.5;` in the parent synthesizes a distinct named branch over the flattened node pair instead of aliasing onto the child's own unnamed branch — the child's flow contribution between the same nodes is no longer discarded by the retention rule (a spurious L022 on fully legal code), and the child's probes read its own branch. Probes in the contributing module over the same hierarchical pair alias onto the contribution's branch (5.5.4's same-module rule); hierarchical references to a child's *named* branch keep merging, which is exactly 5.6.8.2. Upward/absolute paths out of the compiled design (`V(top.drv.a)` from a sibling tree) remain out of scope for the single-design OSDI target — a located resolution error, not a silent drop.

Vector signal-access indices take parameter expressions (LRM 5.5.2): `V(in[width-2])` folds at elaboration; because the index selects a node of the frozen OSDI descriptor, the parameters it reads become structural (frozen to `localparam`, netlist overrides refused with the standard warning) — the same rule parameter-shaped port widths already follow. Genvar and literal-expression indices work as before; an array *variable*'s index stays a runtime access and its parameters stay overridable. ⚠ Reading both the potential and the flow of a never-contributed probe branch — illegal per 5.4.2.1 — is accepted with flow-probe semantics and warning L017 rather than rejected.

5.3 Procedural statements (LRM 5.7–5.9, 5.11) — ✓

`if`/`else`; `case` over integers, reals, strings, and arrays (element-wise); all four loops with runtime trip counts that honor model-card parameter overrides; `disable` as the LRM's named-block early exit; and the VAMS-2023 jump statements `break`/`continue`/`return` (LRM 5.11, Annex-C audit): `break`/`continue` act on the innermost runtime loop of any kind (`for`'s `continue` re-enters at the increment, `repeat`'s still counts the iteration), `return [expr]` exits an analog function — from arbitrarily nested statements — after optionally setting its return value:

```
for (i = 0; i < n; i = i + 1) begin
    if (w[i] == 0.0) continue;          // skip, increment still runs
    if (i == cutoff) break;
    acc = acc + w[i] * x[i];
end

analog function real findk;
    input lim; integer lim; integer k;
    begin
        findk = -1.0;
        for (k = 0; k < 10; k = k + 1)
            if (k == lim) return k;     // early exit, value set
    end
endfunction
```

Position rules are enforced: `break`/`continue` outside a runtime loop — including inside a genvar `analog_for`, which LRM 5.9.3 excludes from jump statements (those loops unroll at elaboration) — and `return` outside an analog function are targeted errors. The keywords are contextual: pre-2023 source that used `break` or `return` as an identifier still compiles, flagged by the L012 keyword-compat lint like the other VAMS reserved words.

Two deliberate relaxations in this clause (behavior audit): ⚠ contribution statements are allowed inside runtime **`repeat`**/**`while`**/ **`for`** loops, which LRM 5.9 forbids (only the genvar `analog_for` may con-

tain them) — the extension is coherent (each executed iteration accumulates, noise handled per E-424) and widely useful, but a model relying on it is not portable LRM Verilog-A; analog filter functions and event controls inside runtime loops stay refused exactly as the LRM demands. ⚠ **`do … while`** parses and runs — a SystemVerilog-style loop that Annex A.6.8's `analog_loop_statement` does not include (`repeat`/`while`/`for` only), supported as a non-LRM extension.

5.4 **`casex`** / **`casez`** — ⚠ extension beyond Annex C

The don't-care case variants are legal in full Verilog-AMS but Annex C.7 excludes them from Verilog-A ("casex and casez are not supported in Verilog-A"); this compiler supports them anyway as a deliberate superset extension, with 2-state semantics: `x`/`z`/`?` digits of a based literal written directly as an item form a comparison mask — the arm matches on every *care* bit. `casex` treats x/z/? as don't-cares, `casez` only z/?. The classic priority-encoder idiom (from the committed `examples/casexz_examples/`):

```
casex (req)
    4'b1xxx: grant = 8;    // highest set bit wins
    4'b01xx: grant = 4;
    4'b001x: grant = 2;
    4'b0001: grant = 1;
    default: grant = 0;
endcase
```

Don't-care literals anywhere else, `x` digits under `casez`, and non-integer discriminants are located errors — no silent zeros.

5.5 Events (LRM 5.10) — ✓

`@(initial_step)` and `@(final_step)` genuinely gate (once at the start; once at the *successful* end, seeing the converged solution), with analysis-phase lists filtering by analysis type; `cross`/`above`/ `timer` with persistent state; event OR lists, with the comma separator the LRM makes interchangeable with `or` (5.10.1):

```
@(initial_step("ac", "tran")) voff = 0.1;
@(cross(V(s) - vth, +1) or timer(0, 1u))
    n_events = n_events + 1;
@(final_step) $strobe("peak = %g at end of run", peak);
```

An operating point fires both step events (a single point is first and last); a failed analysis never fires `final_step`.

The events audit made Table 5-1 exact for every analysis: the operating point of an `.ac`/`.noise` job no longer answers to `"dc"` — the phase belongs to the owning analysis, so `@(initial_step("dc"))` stays silent there and `@(final_step("ac"))` stays silent at the end of a `.noise` run (the same Table 4-22 rows fix `analysis("dc")`/`analysis("ac")` at those points, since both channels share the flag derivation). **`cross`** obeys 5.10.3.2: it "will not generate events for non-transient analyses" and "can only generate an event after the simulation time has advanced from zero" — it used to fire during `.dc` sweeps and off the Newton iterates of the t=0 operating point. Its state still tracks through DC, so the first transient step compares against the converged operating point. **`above`** generates the mandated initialization event: an expression positive at the initial solve fires once (it used to fire only when the Newton trajectory happened to cross the threshold). And the placement rules are enforced: nested event controls are errors ("not allowed" — the nested form was a silently dead statement), `cross`/`above` under a runtime `if`/`case` arm or inside `repeat`/`while`/`for` are errors (their state would go stale; a genvar-conditioned `if` and the genvar `analog_for` stay legal via constant folding and elaboration unrolling), and an analog filter inside the event *expression* is rejected like one in the body always was. An unrecognized event expression is a targeted error — `@(absdelta(…))` (digital-only, 5.10.3.4), named events (5.10.4, outside

the analog subset), and plain typos used to silently DROP the event and run the guarded body on *every* evaluation.

Two deviations, documented: ⚠ **`cross`/`above`** tolerances are accepted but not honored — detection is evaluation-granular and does not bound the timestep, so the event fires at the first solver evaluation past the crossing; a nonzero tolerance now draws a compile-time warning (a 0.0 tolerance means "the simulator shall apply a suitable value", which is exactly what happens — `timer`'s placement, by contrast, is exact via its bound-step channel). ⚠ An out-of-range **`cross`/`last_crossing`** direction is refused (compile error for a literal, runtime fatal from the deck) where LRM 5.10.3.1 defines any other value as a disabled event — E-506's deliberate strictness, kept because a computed direction of 7 is a bug far more often than a disable idiom. Event detection advances once per model *evaluation* (Newton iteration), not per accepted timepoint — the E-7/E-8 granularity design, shared with variable persistence.

5.6 Analog functions (LRM 4.7) — ✓

Declared functions with input/output/inout arguments, arrays in all three roles plus array return values, locals (including array locals), loops inside bodies; inlined at the call site so derivatives flow through automatically:

```
analog function real dot;
    input real a[0:3], b[0:3];
    integer k;
    begin
        dot = 0;
        for (k = 0; k <= 3; k = k + 1)
            dot = dot + a[k]*b[k];
    end
endfunction
```

The UDF audit closed every gap this section used to gloss over: string-typed functions (return and output arguments, 4.7.1/Mantis 7808) work instead of crashing the compiler; function-local **`parameter`** declarations work instead of crashing codegen — a local constant, never an OSDI parameter, with the clause's exact scoping: a local shadows the same-named module parameter, other module parameters read through (and a netlist override of those propagates into the function), `$param_given` on one is constantly false; the LRM's own array-argument spelling compiles — 4.7.1 Example 3 verbatim (`inout [0:1]a; input [0:1]b; real a[0:1], b[0:1];`), the range on the direction line accepted with the dimensions taken from the mandatory data-type declaration (previously the compiler's own name-then-range rewrite generated this very form and then refused to parse it); a pure output array is zero-initialized at entry and an unassigned one resets the caller's array to zeros (4.7.2.3 — it silently had inout copy-in semantics); and the **`return`** statement (4.7.2.2) works, per E-520. Both `$param_given` constant answers are BOOL constants — an integer constant there panicked the MIR folder at the bool→int cast, a latent bug the paramset path shared.

Recursion is illegal in Verilog-A; both direct and mutual recursion are clean errors naming the call cycle (mutual recursion formerly overflowed the compiler stack).

Deliberate relaxations and extensions, recorded: ⚠ named blocks inside function bodies are accepted (4.7.1 forbids them) — pre-2023 code needs `begin : b … disable b` for early exit, the very idiom `return` replaced; ⚠ a UDF call in a constant context (`parameter real p = f(3.0);`) is accepted and constant-evaluated, although 4.7.3 restricts calls to the analog context; ⚠ a formal with no data-type declaration defaults to `real`, the ANSI-style header `f(input real x)` and array return types are OpenVAF extensions beyond Syntax 4-5 (E-389, E-23).

6. Hierarchy (LRM 6) — ✓

Module instantiation with named and positional ports and parameter overrides, open ports, instance arrays, arbitrary nesting, across `include boundaries; **generate** in all three forms with genvars usable in any expression; **defparam** with its LRM precedence over instance overrides — including `defparam` written *inside* a generate block (targeting the per-iteration instance) and module-scope `defparam` coexisting with generate constructs, both of which used to be silently dropped by the generate-rewriting pass; hierarchical names and `$root`; implicit nets; net concatenation in port connections:

```
// genvars usable in any expression, e.g. a parameter override
module gpexpr(a, c);
    inout a, c; electrical a, c;
    genvar i;
    generate
        for (i = 0; i < 3; i = i + 1) begin : g
            res #(.r(1e3*(i+1))) ri (a, c);   // 1k || 2k || 3k
           defparam ri.tc = 1e-4*i;          // per-iteration target: legal (6.3.1)
        end
    endgenerate
endmodule

defparam u1.r = 2e3;              // hierarchical override
defparam u1.u2.r = 4e3;           // two levels down; wins over #(...)
```

Hierarchical system parameters on child instances (LRM 6.3.6): `leaf #(.$mfactor(4)) u1 (p, n);` applies the full multiplicity transform to the inlined child — reads of `$mfactor` compose multiplicatively with the netlist-level instance value (and `$xposition`/`$yposition`/`$angle` additively, `$hflip`/`$vflip` multiplicatively), the child's flow contributions scale by m, its flow probes read the per-copy current back out, and its noise scales as the parallel combination demands (contributed-current noise power $\times m$, contributed-voltage noise power $\div m$). Overrides compose down the hierarchy (`.$mfactor(2)` inside `.$mfactor(4)` is $\times 8$) and with the netlist `m=`. ⚠ Two forms inside a scaled child keep their unscaled shape: indirect contributions (`V(x): I(x) == 0`) and noise routed through a variable rather than contributed directly.

`$param_given` sees hierarchy overrides (LRM 6.3.5/9.19): a child parameter set by an instance `#(...)` value or a `defparam` reports *given*, even though flattening bakes the value in as the parameter's new default (netlist `.model` overrides of the flattened name keep their native OSDI given-flag path).

Connection-size checking (LRM 6.5.7.1): a scalar net — or any actual that does not resolve to a matching-width source — connected to a multi-bit port is a compile error citing the clause (it used to be silently replicated onto every bit). Matching-width buses, part-selects, and `{...}` concatenations connect bit-per-bit as before. Mixing positional and named forms in one `#(...)` parameter list is likewise an error (Syntax 6-2 makes the list all-ordered or all-named; the positional half used to be silently dropped), matching the existing check for port connections.

One boundary the OSDI target makes explicit (and the compiler explains in its diagnostic): structure binds at compile time — a `generate` condition or bound may depend on genvars and literals but not on module *parameters*, because parameters arrive from model cards at simulation time, after structure is frozen. Loop trip counts and `if` arms, being behavior, may depend on parameters freely. ⚠ documented deviation with rationale.

Remaining §6 gaps, all refused with targeted diagnostics rather than silently misbehaving: several generate constructs in one `generate…endgenerate` region (separate regions work), descending or general genvar loops (`i>=0; i=i-1` — only `</<=` with an ascending `+` step unroll), hierarchical references into generate-block instances (`g[2].gg`), `macromodule`, paramset overloading and paramset output variables, and hierarchical access to a child's *port* nets (`V(u1.a, u1.b)`; child internal nets and `$root` paths

work). Two behaviors are ⚠ accepted extensions: named-block variables are readable *and writable* through hierarchical names (the LRM makes the analog-variable assignment an error), and the LRM 6.3.6 double-scaling misuse (a module dividing by `$mfactor` itself while the simulator also scales it) runs without the diagnostic the LRM shows.

7. System tasks and functions (LRM 9)

7.1 Display (LRM 9.4) — ✓

All five kinds (`$strobe`, `$display`, `$write`, `$monitor`, `$debug`) with the complete format surface — `[flags][width][.precision]` on every conversion, dynamic `*` widths, `%e/f/g/r` (engineering notation), `%d/h/o/b/c/s`, `%m`, `%%` — verified against exact printf output. As of E-516 the LRM 9.4.6 timing rule holds: display output is buffered per Newton iteration and printed only for the accepted point (`$debug` keeps the clause's exemption and still prints per iteration; statements inside event-controlled blocks print immediately, since they fire on the event's own iteration). `$monitor` implements 9.4.1's change detection — one line per change of its argument list at accepted points (a `$abstime` argument defeats the text comparison — documented). `%r` engineering notation, which printed garbage for every input, is exact: `1e3` → `1.000000k`, `1e-9` → `1.000000n`, `0.036` → `36.000000m`. A null argument (two adjacent commas) renders as the single space 9.4.1 specifies. A constant-argument display hoisted to instance initialization still executes there (⚠ documented at the split):

```
$strobe("Vth=%7.4f V  region=%2d  id=%r  name=%s", vth, reg, id, name);
```

7.2 File and string I/O (LRM 9.5, 9.8) — ✓

`$fopen/$fclose/$fdisplay/$fwrite/$fstrobe/$fmonitor/$fdebug/$fflush/ $ftell/ $fseek/$rewind/$feof/$ferror/$fgetc/$ungetc` (single-character read and one-character pushback, E-107/E-108) and `$swrite/$sformat/$sscanf/ $fgets/$fscanf`. The 9.5 lifecycle rules hold as of E-516: un-gated file writes are deferred to the accepted iteration (9.5.9, with readable streams rewound to their accepted baseline each iteration for the read replay); a `"w"`-mode reopen in a following analysis appends (9.5.1.1); the open-write-close idiom in the analog body writes its line (an instance-setup `$fclose` defers so the descriptor survives for eval); a re-run of the instance initialization reproduces `$rewind`/`$fseek` files byte-exactly; and the pre-opened descriptors `32'h8000_0000/1/2` reach stdin/stdout/stderr. Full one-hot multichannel descriptors and `$fmonitor` change detection remain ⚠ documented gaps. `$sscanf`/`$fscanf` honour the format conversion base — `%h`/`%x` hex, `%o` octal, `%b` binary, `%d` decimal (E-105).

The scan format is read only when it is a literal (E-507). The scanners are chosen at compile time from the format text, so a format that is not a literal cannot select them: such a call used to read the *text of the format itself* as data and report success. It is refused now, as every other builtin needing a compile-time string already did. Within a literal format, the parts that the scanners cannot honour are refused with a reason rather than silently ignored — literal characters between conversions, a field width, `%*` assignment suppression (which returned the very field it asks to discard), and unsupported conversion characters. A conversion that does not match leaves its destination unchanged rather than zeroing it, so a scan into a variable that already holds a value is non-destructive:

```
integer fd;
analog @(final_step) begin
    fd = $fopen("iv.dat");
    $fdisplay(fd, "%g %g", vpeak, ipeak);
    $fclose(fd);
end
```

7.3 Random and distributions (LRM 9.13) — ✓ (deterministic semantics)

`$random`, `$arandom`, and every `$dist_*`/`$rdist_*` (uniform, normal, exponential, poisson, chi-square, t, erlang), verified against closed-form moments. ⚠ One deliberate semantic: draws are a deterministic function of *(seed, call site)* with no in-place seed advance — the LRM 9.13.1 inout seed would return different values on every Newton iteration and destroy convergence. The seed VARIABLE is never written, so successive calls at one call site with one seed value return the identical number (an in-model sampling loop collects N copies of one deviate). This gives reproducible Monte Carlo and independent per-instance variation, at the cost of in-evaluation sequences (which have no convergent meaning in an analog solver anyway).

Domain rules are errors on every route (kernel audit): 9.13.2's "mean, degree_of_freedom, and k_stage shall be greater than zero. Otherwise an error shall be reported" was only enforced for arguments the compiler could see; a deck-supplied violation silently clamped (exponential, poisson) or passed straight to the RNG (chi-square, t, erlang — a *negative* deviate from a distribution supported on [0, ?)). A deck-derived violation now aborts with the mandated runtime error naming the function, argument and clause; the E-505/506 clamps remain as the safety net for values the guard cannot attribute to a parameter. The `type_string` argument (`"global"`/`"instance"`) warns outside a paramset, where 9.13.1/9.13.2 scope it and where it has no effect.

Return types follow the LRM split: `$dist_*` returns `integer` and `$rdist_*` returns `real` — the `$rdist_*` family exists precisely because the originals, inherited from Verilog-2001 §17.9.2, are integer-only. Both returned `real` until E-376, which made the LRM-conformant spelling `$display("%d", $dist_uniform(s,10,20))` compile; it had been rejected with *"expected integer value but found real value"*. The draws themselves did not move — only the static type — and the discrete/continuous split is visible in the moments: `$dist_uniform` sits at $(n^2-1)/12$ while `$rdist_uniform` sits at 100/12.

```
parameter integer seed = 1;
real gain;
analog begin
    gain = 1.0 + sigma * $rdist_normal(seed, 0.0, 1.0);  // per-instance mismatch
    ...
end
```

7.4 Simulation control (LRM 9.6–9.7) — ✓

`$finish`, `$stop`, `$fatal` are honored with simulator-correct timing (at the accepted-point boundary; `$finish` fires `@(final_step)` and ends the analysis cleanly; `$fatal` during setup reads as a configuration rejection), and `$warning`/`$error`/`$info` display.

7.5 The mechanism-unavailable group — ⚠

`$simprobe(inst, quant, default)` returns its supplied default — the LRM 9.16 fallback for a host that resolves no probes — and, since the kernel audit, the no-default form is a compile-time warning plus a runtime fatal, which is exactly the error 9.16 mandates there (it used to return 0.0 in silence, which this section previously mis-described as an LRM fallback). `$analog_node_alias`/`$analog_port_alias` return 0 (no alias created — the clause's unresolvable-reference value), and their 9.20 context rule is enforced: a call outside an `analog initial` block is a compile error. (`$test$plusargs`/`$value$plusargs` are NO LONGER in this group: Enhancement-215 serves command-line plusargs through the simparam channel.) This is the LRM's own escape hatch for hosts without the mechanism; models using them run predictably instead of being rejected.

7.6 Attributes as simulator metadata — ✓

`(* desc="…" *)` on a module-scope variable exposes it as an operating-point variable (`print @n1[gm]`, `.save`, `.meas`) — a described variable declared inside a named block is local to that block (LRM 3.2.1) and is no longer exported as a phantom op-var; `(* type="instance" *)` on a parameter puts it on the instance line and under `alter`/`.dc` sweeps; `desc`/`units` on parameters populate the OSDI descriptor.

8. Compiler directives (LRM 10) — ✓

The predefined source-location macros work (E-85): `` `__FILE__ `` expands to the source file's basename, `` `__LINE__ `` to the exact line (a use inside a `` `define `` body reports the definition site — documented textual-expansion semantics):

```
$strobe("evaluated at %s:%0d", `__FILE__, `__LINE__);
```

`` `define `` with arguments (macros-in-macros, macro calls as arguments), `` `ifdef ``/`` `ifndef ``/`` `elsif ``/`` `else ``/`` `endif `` chained and nested, `` `include `` chains, `` `undef `` (which, per LRM 10.4, has no effect on the predefined macros and warns — E-515, which also makes `` `__VAMS_ENABLE__ `` always-defined per 10.5, validates `` `begin_keywords ``/`` `end_keywords `` version specifiers per 10.6, and fixes `` `__FILE__ ``/`` `__LINE__ `` breaking relative `` `include `` resolution), `` `resetall `` (now genuinely resetting the tracked directive state), `` `default_transition ``, and the housekeeping set (`` `celldefine ``, `` `timescale ``, `` `line ``, `` `pragma ``, `` `default_nettype ``, …). Recursive macro expansion — direct or mutual — is a clean located error (formerly a compiler stack overflow), while the legal same-macro nesting inside *arguments* (`` `QUAD(x) = `TWICE(`TWICE(x)) ``) expands finitely:

```
`define TWICE(x) ((x)*2.0)
`define QUAD(x) (`TWICE(`TWICE(x)))   // same-macro nesting in ARGUMENTS: legal
`ifdef HIGH_PRECISION
    `define TOL 1e-12
`else
    `define TOL 1e-9
`endif
```

(`` `default_discipline `` is AMS-only and out of scope — implicit nets themselves are Verilog-A and work, taking their discipline from the connected port.)

9. Compliance summary

LRM area	Status	Verified by
2 Lexical (identifiers, numbers, strings, attributes, three-token based literals, expression-position attrs)	✓ (E-515 audit)	`escid`, `stresc`, `comment`, `lrmlex`
3.2–3.3 Value types, variables, persistence	✓	`vartype`, `variable_persistence`, `intstate`, `varinit`
3.4 Parameters (ranges, localparam, aliasparam incl. 3.4.7 error rules, arrays incl. whole-array instantiation override, paramset, block-scoped)	✓ (default-exemption ⚠ and frozen-type-of-untyped ⚠ documented)	`paramrange`, `localparam`, `array`, `paramarray`, `paramset`, `paramsethsp`, `blockparam`, `alias`

LRM area	Status	Verified by
3.5–3.13 Natures, disciplines, nets, buses, nodesets, 3.11.1 compatibility (signal-flow/ natureless/domainless), nature-attribute validation, NIST2018 constants	✓ (base-nature attr omission ⚠ and derived-access extension ⚠ documented; vector branches 🚫 rejected)	`derivednature`, `domainbind`, `bus`, `netinit`, `ground`, `signalflow`
4.1–4.4 Operators (incl. string relational, `===`/`!==` as 2-state `==`/`!=`), precedence, functions (`$clog2`, `$rtoi`/`$itor`, `ln1p`/`expm1`, domain diagnostics both routes, `%`-by-deck-zero fatal, same-node `V(a,a)`/`I(a,a)` rejected)	✓ (`<<<`/`>>>` extension ⚠ warned; runtime-probe domain policy ⚠ documented)	`operator`, `precedence`, `shift`, `concat`, `stringcmp`, `clog2`, `convert`, `ceil`, `lrmfuncs`, `lrmintrin`, `domainrt`
4.5 Analog operators (all), clause-by-clause audit 4.5.1–4.5.15 (E-514); frozen-`td` absdelay implemented, ddx over unnamed-branch flows, no-ic `idtmod` pins DC at 0, `default_transition` honored for explicit zeros	✓ (`limexp` stateless ⚠, idt-reset τ=10µs ⚠, ddt/idt tolerances not plumbed ⚠, transition amplitude approximation ⚠, zi continuous-bilinear ⚠, dynamic laplace coeffs track+warn ⚠ — all documented)	`absdelay`, `laplace`, `zi`, `slew`, `transition`, `idt*`, `ddx`, `opargs`, `discontinuity`, `last_crossing`, `defaulttransition`, `transedge`, `rtdomain`, `deckdomain`
4.6 Noise & analysis (correlation per call site with label-combined reporting 4.6.4.1/4.6.4.6; `noise_table` linear-in-f, `noise_table_log` log-log, parameter-fed tables ⚠ refused; Table 4-22 ic/static/ nodeset phases exact; ac_stim matched to the running small-signal analysis)	✓	`noise`, `noisetable`, `noisecorr`, `noisejw`
5.2/6.2 Analog blocks (multiple)	✓	`multianalog`
5.6 Contributions, indirect assignment (placement rules and the 5.6.7.2 direct/indirect incompatibility enforced; generalized equality LHS ⚠ extension), hierarchical-contribution branch identity (5.6.8.1), parameter vector indices (5.5.2), port flow (incl. named port branches)	✓	`indirect_assignment`, `portflow`, `signalflow`, `lrm`
5.7–5.9, 5.11 Procedural statements, loops, disable, jump statements (`break`/`continue`/`return`, contextual keywords, genvar-for exclusion enforced)	✓ (contributions in runtime loops ⚠ and `do…while` ⚠ are documented non-LRM extensions)	`analogloop`, `dowhile`, `repeat`, `disable`, `arraycase`
casex/casez	⚠ extension beyond Annex C (C.7 excludes them from Verilog-A)	`casexz`

LRM area	Status	Verified by
5.10 Events (steps with Table 5-1 exact per analysis, cross/above/ timer, cross DC/t=0 gating, above init event, OR lists incl. comma, phase lists, placement rules enforced, invalid events rejected)	✓ (cross/above tolerances ⚠ warned not honored; strict out-of-range dir ⚠ documented)	`initial_step`, `finalstep`, `cross`, `timer`, `lrmcorner`
4.7 Analog functions (arrays in/ out/return incl. the LRM's Example 3 spelling; VAMS-2023 string return & args; function-local `parameters` with shadowing; output-array zero-init; `return` statement)	✓ (named blocks in bodies ⚠ and const-context calls ⚠ documented relaxations)	`funcarray`, `arrayout`, `arrayret`
6 Hierarchy (instantiation, generate incl. the legacy analog-block form — obsolete per G.2.3/C. 20, kept as a compat extension ⚠ — defparam incl. inside/beside generate, `#(.$mfactor(n))`-family child overrides with the full multiplicity transform, `$param_given` through hierarchy overrides, connection-size and mixed-override checking, $root, part-select connections, hierarchical branch probes)	✓ (param-shaped structure ⚠ explained)	`instantiation`, `generate`, `legacygen`, `defparam`, `hiername`, `implicitnet`, `partselect`, `hierbranch`
9.4–9.8 Display, file/string I/O (incl. 9.4.6/9.5.9 accepted-iteration deferral, `$monitor` change detection, `%r`, 9.5.1.1 append, pre-opened fds)	✓ (E-516 audit; MCD/`$fmonitor` ⚠)	`display`, `fileio`, `stringio`, `fgetc`, `ungetc`, `sscanf`, `scanfmt`, `lrmsysio`
9.13 Random/distributions (9.13.2 domain rules errors on the deck route too; `type_string` scoped to paramsets)	✓ (deterministic seed ⚠ documented)	`rng`, `montecarlo`
9.17.3 / Annex E.3.4 `$limit`	✓ (unknown name falls back to no limiting with a warning, per 9.17.3; `vdslim` = Table E.2's preferred spelling of `limvds`)	`fetlim`, `lrm`
Annex E SPICE primitives (E.2/E.3, Table E.1)	? not supported: `resistor`/`capacitor`/`bjt`/... cannot be instantiated from Verilog-A source (clean error); instantiate SPICE devices at netlist level instead	—

LRM area	Status	Verified by
9 misc ($finish family, $simparam; *simparam*str serves analysis_name/analysis_type/cwd/simulator — module/instance/path unserved ⚠; $bound_step smallest-wins; $table_model per 9.21 incl. isoline files, per-dimension controls, D/E codes, `;N` selector — 2/I/whole-array-N-D ⚠ refused; attributes)	✓	`simctrl`, `simparamstr`, `opvar`
plusargs (`$test`/`$value`)	✓ (E-215)	`plusargs`
simprobe / node aliases	⚠ LRM fallbacks (no-default `$simprobe` = warn + the mandated runtime fatal; aliases analog-initial-only, enforced)	`alias`
10 Compiler directives (incl. `` `__FILE__ ``/ `` `__LINE__ ``, predefined-macro protection, `` `begin_keywords ``)	✓ (E-515)	`preproc`, `directive`, `defaulttransition`, `filemacro`, `lrmlex`
Mixed-signal / digital (outside Annex C)	× by scope	—

As of E-84 the standard's own examples are a compliance suite: every code example in the LRM 2023 PDF is extracted and compiled (`examples/lrm_examples/` — per its `manifest.json`: 47 compile, 12 documented limitations pinned to their diagnostics, 21 correctly rejected as mixed-signal), and the 146 non-module fragments double as a no-crash fuzz corpus.

Two later findings are worth recording here because both were *compliance* failures rather than robustness ones — legal Verilog-A that the compiler refused or mishandled, found only after the construct-level suites above were all green:

- Array parameters could not survive instantiation (E-363). A module declaring `parameter real cf[0:2]` could not be instantiated twice: elaboration renamed scalar parameters and array *variables* per instance but not array *parameters*, so both instances re-declared `cf[0]`, `cf[1]`, ... and name resolution rejected the second. The same collision hit two *different* modules that merely shared an array-parameter name. This sits across LRM 3.4 (parameters) and LRM 6 (hierarchy), and no single-feature test could see it — it needs a parameter form and instantiation together.
- A **`case`** inside a **`do-while`** crashed the compiler (E-363) — LRM 5.7/5.9 constructs that are individually covered by the `arraycase` and `dowhile` suites, but had never been *composed*.
- Two standard functions crashed the compiler, while passing their own test (E-510). `ln1p` and `expm1` (LRM Table 4-14) lower to libm routines the code generator requests by name, and the intrinsic registry declared neither — so every call whose argument was not constant-folded away aborted the compiler. The suite that introduced them tested `$ln1p(0.5)`, and a literal folds before code generation, so the intrinsic was never emitted and the test could not reach the broken path. This is worth recording as a methodology point rather than a bug: for anything that lowers to a call, a literal argument does not exercise code generation, and a construct-level suite written only with literals can be green over a feature that no user can use. The fix was two declarations; finding it took fuzzing 2 089 constant-folded expressions and noticing that exactly two crashed for *every* argument.

Both of the first two came from a generator that emits valid-by-construction programs combining fea-

tures developed in isolation, which is a different instrument from the mutation fuzzing used earlier: mutants die at the parser, so they never reach the semantics. One defect from the same round was left open at the time — a provably non-terminating analog loop — and is now closed by E-375: it is a compile-time error. There is no correct object code for a model that cannot complete one evaluation, so a diagnostic was always the only right answer. Worth recording why it could not simply be left: the E-363 CFG repair had stopped the compiler crashing and started it *emitting*, and the resulting `.osdi` loaded cleanly and then hung the simulator on the first device evaluation with no diagnostic at all — strictly worse than the crash.

Separately, the whole 496-file corpus now compiles with zero assertion failures under an assertions-enabled build (E-347), which is a stronger statement than the release-build corpus run: a release build compiles every `debug_assert!` out, so internal-invariant violations pass silently there.

Beyond construct-level checks, three cross-cutting guards pin that the *whole language implementation* produces correct physics: the 92-model industry corpus compiles and runs, the static/dynamic/thermal physics suites recover textbook device laws through the full pipeline (60 mV/dec, C $\equiv$ dQ/dV across analyses, E_g $\approx$ 1.25 eV from a compiled MEXTRAM), and the twin benchmarks show OSDI output matching hand-coded simulator devices to machine precision.